

DP Computer Science: B 2.2.1 Static and Dynamic Data Structures

1. Core Concepts & Learning Objectives

Learning Intentions

By the end of this lesson, students will be able to:

- **Differentiate** between static and dynamic data structures
 - Explain how **memory allocation** and **resizing** differ between the two types
 - **Evaluate** the advantages and disadvantages of each type in different scenarios
-

Key Vocabulary

Term	Definition
Static data structure	A data structure with a fixed size that cannot be changed at runtime
Dynamic data structure	A data structure that can grow or shrink as needed during runtime
Memory allocation	The process of reserving memory for data storage
Compile-time allocation	Memory is allocated when the program is compiled (before execution)
Runtime allocation	Memory is allocated while the program is running
Stack	A region of memory used for static allocation (fast, fixed size)
Heap	A region of memory used for dynamic allocation (flexible, but slower)

Term	Definition
Resizing	The ability to change the size of a data structure

ATL Skills

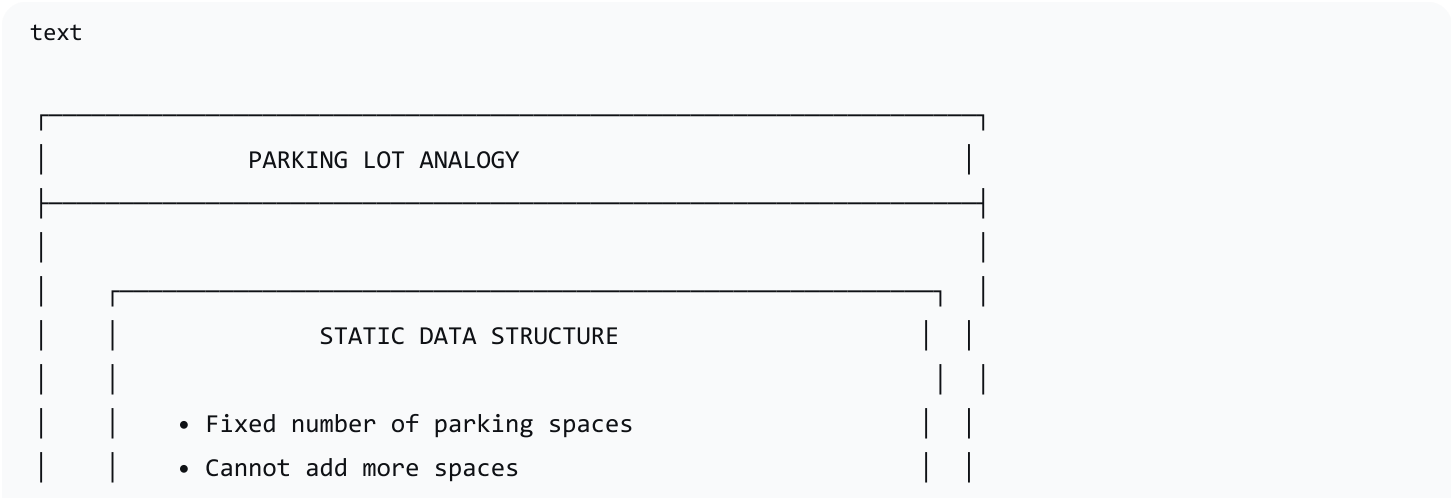
Skill	Description
Thinking	Critical thinking (analysing advantages and disadvantages); application (selecting appropriate structures)
Communication	Explaining differences and justifying choices
Self-Management	Organising understanding through comparison and evaluation

2. Introduction: The Tool Analogy

The Wrench Analogy

"Choosing between static and dynamic data structures is like choosing the right tool—sometimes you need a fixed-size wrench, and sometimes an adjustable one."

Real-World Analogy: Parking Lot vs. Valet Parking



- If full, no more cars can park
- Fast to find a specific car (direct access)
- Analogy: A parking lot with fixed spaces

DYNAMIC DATA STRUCTURE

- No fixed number of parking spaces
- Can add more spaces as needed
- Never runs out of space
- Slightly slower to find a specific car
- Analogy: Valet parking (spaces are allocated and freed as needed)

3. Static Data Structures

Definition

A static data structure has a fixed size that is determined at compile time and cannot be changed during program execution.

Memory Allocation

Aspect	Description
Allocation Time	Compile-time (before the program runs)
Memory Location	Stack (fast, but limited)
Size	Fixed; cannot be changed
Flexibility	None

Example: Array

text

STATIC ARRAY EXAMPLE

```
int[] scores = new int[5];
```

Index:	0	1	2	3	4
Value:	78	85	92	63	89

✓

Fixed size of 5 elements

✗

Cannot add a 6th score

✗

Memory is wasted if only 3 scores are stored

✓

Fast access to any element (direct indexing)

Characteristics of Static Data Structures

Characteristic	Description
Size	Fixed at creation; cannot grow or shrink
Memory	Allocated at compile time
Location	Stored on the stack
Access Speed	Very fast (direct indexing)
Memory Efficiency	Efficient if size is known; wasteful if over-allocated
Flexibility	Not flexible; cannot handle varying data sizes
Risk	Overflow if too much data is stored; wasted memory if too little

Advantages of Static Data Structures

Advantage	Description
Fast Access	Direct indexing provides O(1) access
Efficient Memory	No overhead for pointers or dynamic allocation
Predictable	Fixed size means no unexpected growth
Simple	Easy to implement and use

Disadvantages of Static Data Structures

Disadvantage	Description
Inflexible	Cannot grow or shrink
Memory Waste	May allocate more memory than needed
Overflow Risk	May not have enough memory for all data
Poor for Unknown Sizes	Not suitable for data that changes in size

Common Static Data Structures

Structure	Description	Use Case
Array	Fixed-size collection of elements	Known number of items (e.g., days of the week)
Static Stack	Fixed-size LIFO structure	Limited stack depth
Static Queue	Fixed-size FIFO structure	Bounded buffer

4. Dynamic Data Structures

Definition

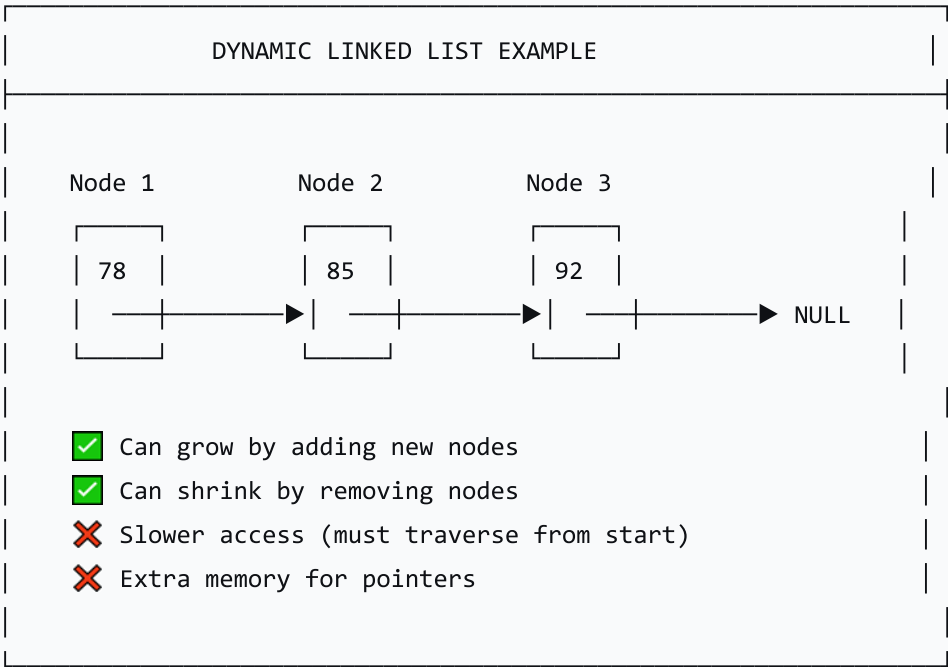
A dynamic data structure can grow or shrink as needed during program execution, allocating memory at runtime.

Memory Allocation

Aspect	Description
Allocation Time	Runtime (while the program is running)
Memory Location	Heap (flexible, but slower)
Size	Flexible; can grow or shrink
Flexibility	High

Example: Linked List

text



Characteristics of Dynamic Data Structures

Characteristic	Description
Size	Flexible; grows and shrinks at runtime
Memory	Allocated at runtime on the heap
Location	Stored on the heap
Access Speed	Slower (requires traversal)
Memory Efficiency	Efficient for varying data sizes
Flexibility	Highly flexible; adapts to data
Risk	Memory fragmentation; pointer overhead

Advantages of Dynamic Data Structures

Advantage	Description
Flexible	Can grow or shrink as needed
Efficient Memory	Only uses memory for data that exists
No Overflow	Never runs out of space (within heap limits)
Adaptable	Suitable for unknown or changing data sizes

Disadvantages of Dynamic Data Structures

Disadvantage	Description
Slower Access	No direct indexing; traversal required
Memory Overhead	Extra memory for pointers/references
Complexity	More complex to implement and manage

Disadvantage	Description
Memory Fragmentation	Can lead to fragmented memory over time

Common Dynamic Data Structures

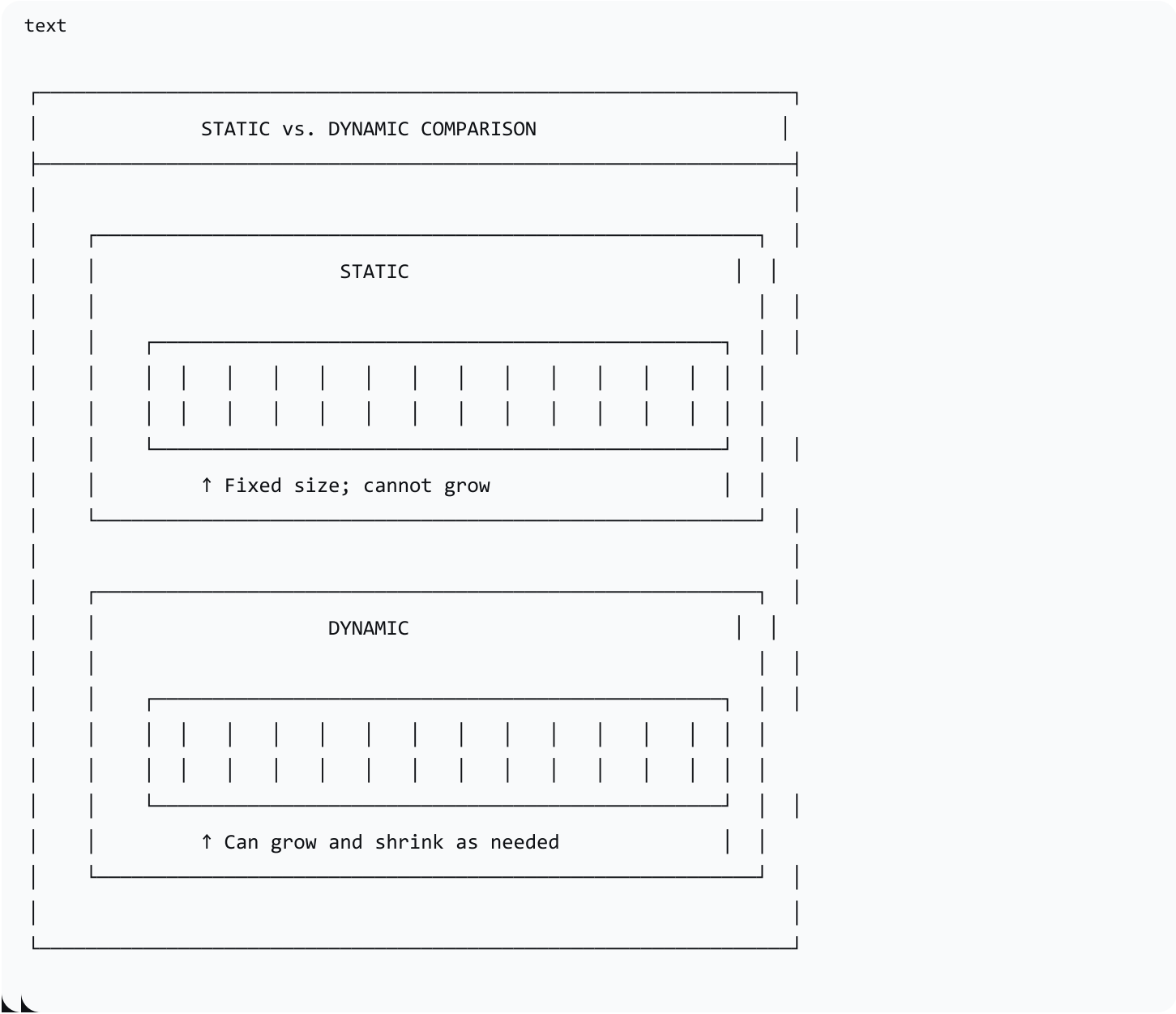
Structure	Description	Use Case
Linked List	Sequence of nodes with pointers	Unknown number of elements
Dynamic Stack	Growable LIFO structure	Stack with variable depth
Dynamic Queue	Growable FIFO structure	Queue with variable length
Tree	Hierarchical structure with dynamic nodes	Hierarchical data
Graph	Network of connected nodes	Complex relationships
ArrayList (Python list)	Resizable array	General-purpose dynamic collection

5. Comparison Table

Feature	Static	Dynamic
Size	Fixed	Flexible
Memory Allocation	Compile-time	Runtime
Memory Location	Stack	Heap
Resizing	Not possible	Possible
Access Speed	Fast ($O(1)$)	Slower ($O(n)$ for traversal)
Memory Efficiency	Efficient if size is known	Efficient for varying sizes

Feature	Static	Dynamic
Overhead	None	Pointer/reference overhead
Complexity	Simple	More complex
Overflow Risk	Yes	No
Fragmentation	None	Possible
Examples	Array, Static Stack, Static Queue	Linked List, Dynamic Stack, Tree

Visual Comparison



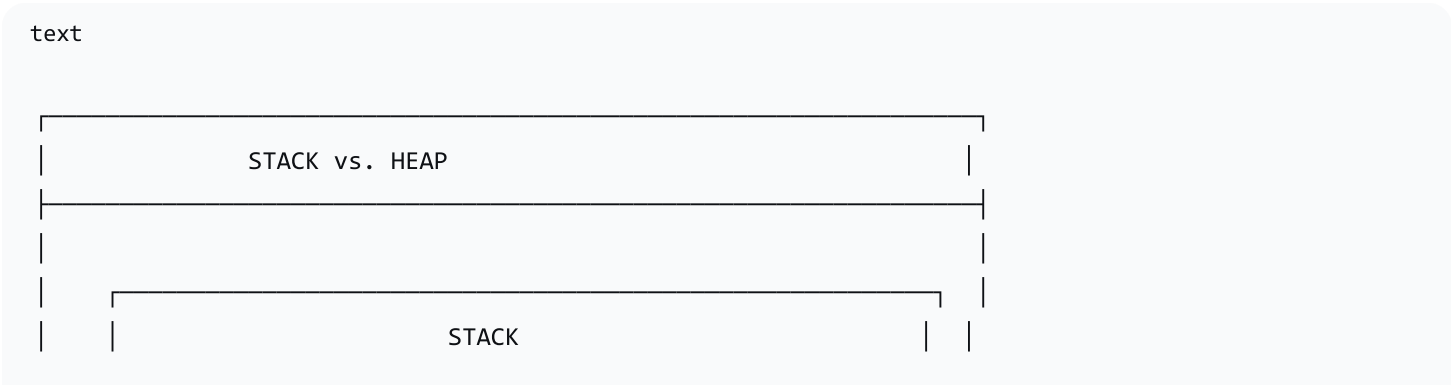
6. Memory Allocation: Stack vs. Heap

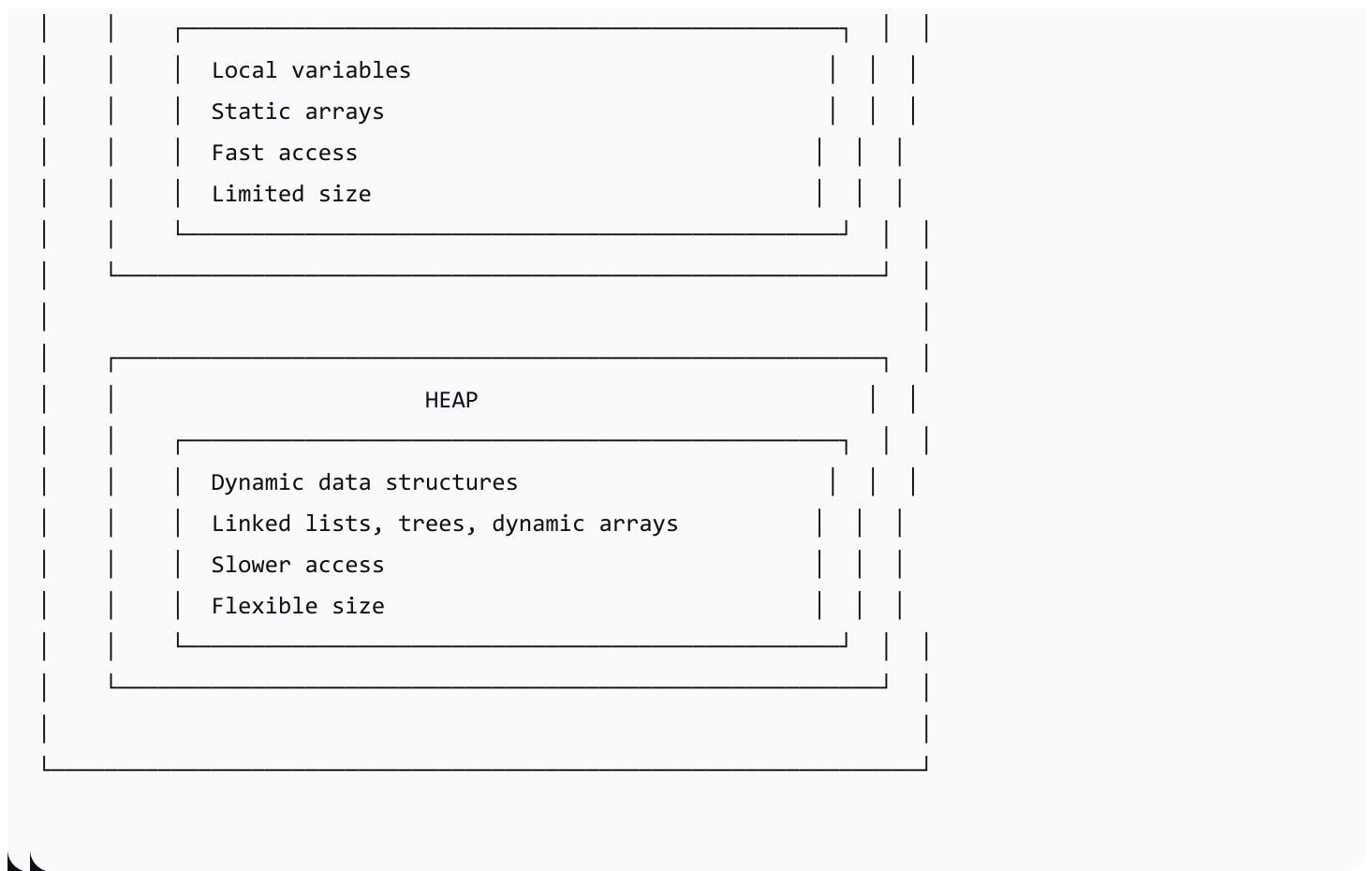
The Stack

Aspect	Description
Allocation	Compile-time (static)
Speed	Very fast
Size	Fixed and limited
Memory Management	Automatic (LIFO)
Used For	Static data structures, local variables

The Heap

Aspect	Description
Allocation	Runtime (dynamic)
Speed	Slower than stack
Size	Flexible and larger
Memory Management	Manual or garbage collected
Used For	Dynamic data structures, large objects





7. When to Use Static vs. Dynamic

Use Static When...

Scenario	Why?
Known data size	You know exactly how much data will be stored
Performance is critical	Static structures are faster
Memory is limited	No overhead for pointers
Data size is constant	The amount of data will not change
Examples	Days of the week, months of the year, fixed arrays

Use Dynamic When...

Scenario	Why?
Unknown data size	You don't know how much data will be stored
Data changes frequently	Elements are added and removed often
Memory efficiency	Only use as much memory as needed
Flexibility is important	Need to adapt to changing requirements
Examples	User input, inventory, social media feeds

8. Scenario Cards Activity

Instructions

1. In groups, analyse each scenario
2. Determine whether a static or dynamic data structure would be more suitable
3. Justify your choice based on:
 - The amount of data to be stored
 - Whether the amount of data will change frequently
 - The importance of memory efficiency
 - The need for fast access to data

Scenario 1: Storing Student Information

Problem: A school needs to store information for 500 students. The number of students is known and changes rarely.

Analysis	Recommendation
Data Size	Known (500 students)
Changes	Rarely
Performance	Important (fast access needed)

Analysis	Recommendation
Memory	Efficient memory use
Recommended: Static (Array)	
Why: The number is fixed, and fast access is important.	

Scenario 2: Managing an Online Store Inventory

Problem: An online store needs to track inventory. Products are added and removed frequently.

Analysis	Recommendation
Data Size	Unknown; changes frequently
Changes	Frequent additions and removals
Performance	Moderate
Memory	Efficiency is important
Recommended: Dynamic (Linked List)	
Why: Inventory changes frequently; flexibility is important.	

Scenario 3: Tracking Social Media User Data

Problem: A social media platform needs to store user profiles for millions of users. New users join every day.

Analysis	Recommendation
Data Size	Unknown; grows every day
Changes	Constant additions
Performance	Moderate
Memory	Important to use only what's needed

Recommended: Dynamic (Linked List / Tree)

Why: User base grows constantly; need flexible storage.

Scenario 4: Days of the Week

Problem: A program needs to store the days of the week.

Analysis	Recommendation
Data Size	Fixed (7 days)
Changes	Never
Performance	Important
Memory	Efficient

Recommended: Static (Array)

Why: Fixed size; fast access; no need for flexibility.

9. Lesson Sequence

Lesson Plan (60 minutes)

Phase	Time	Activity
Engage	10 min	Parking lot analogy; brainstorm data structures
Explore	25 min	Scenario cards: groups discuss and categorise
Explain	25 min	Interactive lecture: definitions, memory allocation, advantages/disadvantages
Assessment	—	Quiz on eBook; written assignment

10. Assessment Activities

Assessment Type	Description
Class Participation	Observe engagement and contributions
Scenario Analysis	Evaluate ability to choose appropriate structures
Quiz	eBook page B2.2.1 quiz

Sample Quiz Questions:

1. What is the main difference between static and dynamic data structures?
2. Where is memory allocated for static data structures?
3. Why is access faster in static data structures?
4. Give an example of a dynamic data structure.
5. When would you use a static data structure over a dynamic one?

11. Differentiation

Level	Strategy
Support	Provide visual diagrams and real-world analogies
Challenge	Explore implementation details (how dynamic structures grow)

12. Resources

- Presentation (Slide Deck)
- eBook (B2.2.1 with quiz)
- Quizlet (B2.2.1 vocabulary flashcards)
- Activity: Scenario cards

13. Summary: Key Takeaways

Concept	Key Point
Static	Fixed size; compile-time allocation; fast access
Dynamic	Flexible size; runtime allocation; adaptable
Stack	Memory location for static structures
Heap	Memory location for dynamic structures
Array	Common static data structure
Linked List	Common dynamic data structure

text

KEY REMINDERS

- ✅ Static = fixed size, compile-time, stack, fast
- ✅ Dynamic = flexible, runtime, heap, slower
- ✅ Static = good for known, fixed data
- ✅ Dynamic = good for unknown, changing data
- ✅ Choice depends on data size, changes, performance

"Choosing between static and dynamic data structures is about balancing flexibility, performance, and memory efficiency."